Array

Lecture 12

Based on Slides of Dr. Norazah Yusof

Introducing Arrays

- **Array is a data structure that represents a collection of the same types of data.**
- **In Java, array is an object that can store a group of values, all of the same type.**

Declare and create an array

- **Declare and create array is similar to create and create any other type of object.**
- **Example question:**

  Declare and create 10-element double array referenced by the variable, myList.

Declare and create an array in two steps

Step 1: declare an array reference

```
datatype[] arrayRefVar;
```

or

```
datatype arrayRefVar[];  // This style is allowed,
                         // but not preferred
```

Step 2: create an array

```
arrayRefVar = new datatype[arraySize];
```

Example

```
double[] myList;
myList = new double[10];
```

Declare and create an array in one step

- **Declare and create the array**

```
datatype[] arrayRefVar = new datatype[arraySize];
```

or

```
datatype arrayRefVar[] = new datatype[arraySize];
```

- **Example**

```
double[] myList = new double[10];
```

or

```
double myList[] = new double[10];
```

The Length of an Array

Once an array is created, its size is fixed. It cannot be changed. You can find its size using

```
arrayRefVar.length
```

For example,

```
myList.length
```

returns 10

Default Values

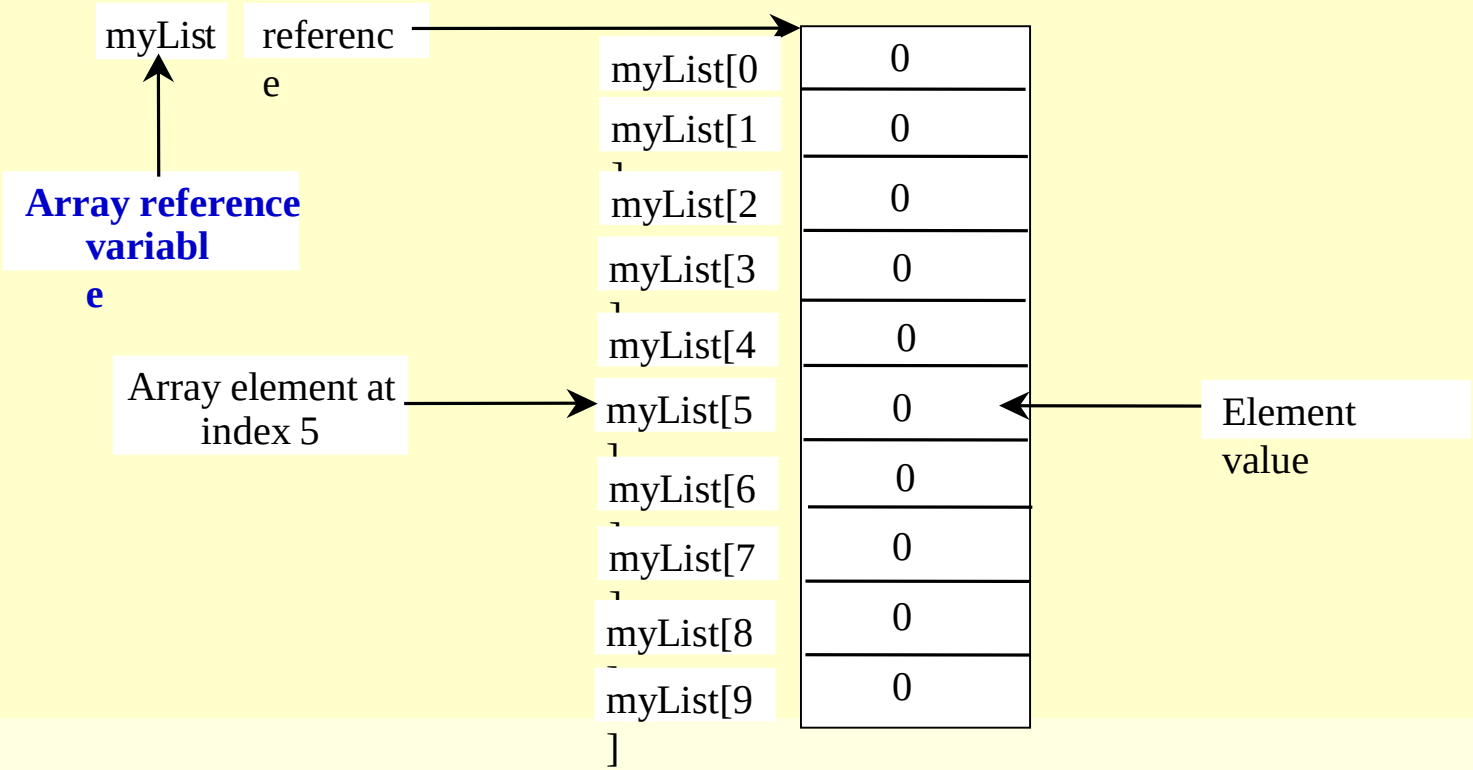
When an array is created, its elements are assigned the default value of

- `0` **for the numeric primitive data types,**
- `'\u0000'` **(unicode value) for <u>char</u> types, and**
- `false` **for <u>boolean</u> types.**

Arrays

- **Subscript**
  - **Integer contained within square brackets**
  - **Indicates one of array's variables or elements**

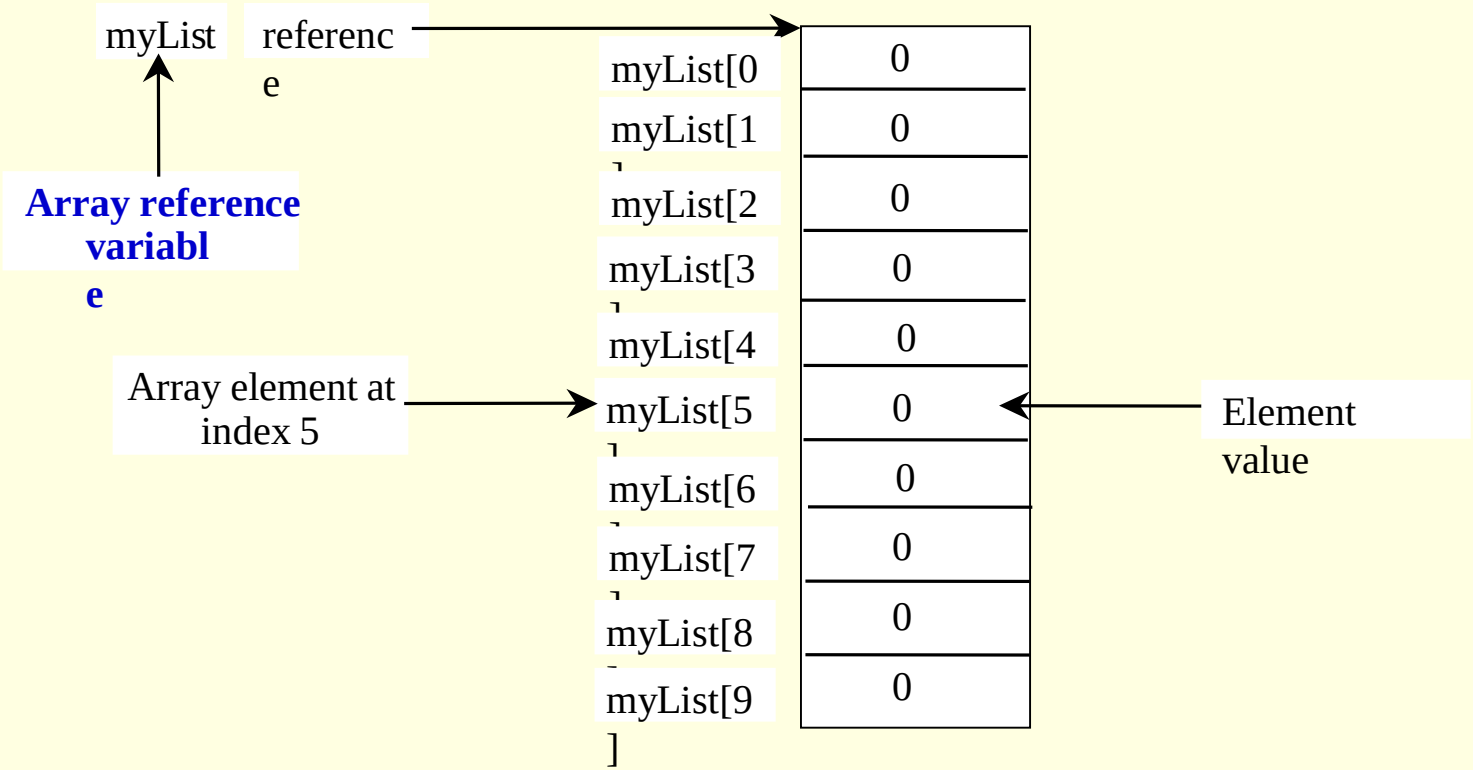
```
double[] myList = new double[10];
```



Arrays

- **Array's elements numbered beginning with zero**
  Can legally use any subscript from 0 through `myList.length-1`

```
double[] myList = new double[10];
```



Accessing Array Elements

- **The array elements are accessed through the index, known as indexed variable**

```
arrayRefVar[index];
```

- **An indexed variable can be used in the same way as a regular variable.**
- **For example, the value of the array element may be assigned as follows:**

```
myList[0] =  5.6;
myList[1] =  myList[0] - 1.1;
Scanner inp = new Scanner(System.in);
myList[2] = inp.nextDouble();
```

Accessing Array Elements

- **The array elements can be output using the println method.**

```
System.out.println (myList[0]);
System.out.println (myList[1]);
System.out.println (myList[2]);
```

Write & Trace the output

```
class ArrayDemo {
    public static void main(String[] args) {
        int[] anArray; // declares an array of integers
        anArray = new int[10]; // allocates memory for 10 integers
        anArray[0] = 100; // initialize first element
        anArray[1] = 200; // initialize second element
        anArray[2] = 300; // etc.
        anArray[3] = 400;
        anArray[4] = 500;
        anArray[5] = 600;
        anArray[6] = 700;
        anArray[7] = 800;
        anArray[8] = 900;
        anArray[9] = 1000;
```

```
System.out.println("Element at index 0: " + anArray[0]);
System.out.println("Element at index 1: " + anArray[1]);
System.out.println("Element at index 2: " + anArray[2]);
System.out.println("Element at index 3: " + anArray[3]);
System.out.println("Element at index 4: " + anArray[4]);
System.out.println("Element at index 5: " + anArray[5]);
System.out.println("Element at index 6: " + anArray[6]);
System.out.println("Element at index 7: " + anArray[7]);
System.out.println("Element at index 8: " + anArray[8]);
System.out.println("Element at index 9: " + anArray[9]);
    }
}
```

- **The output from this program is:**

```
Element at index 0: 100
Element at index 1: 200
Element at index 2: 300
Element at index 3: 400
Element at index 4: 500
Element at index 5: 600
Element at index 6: 700
Element at index 7: 800
Element at index 8: 900
Element at index 9: 1000
```

Using loop to cycle through the entire array

- **Perform loops that vary loop control variable**
  - **Start at 0**
  - **End at one less than size of array**
- **Using constant that is equal to size of array**

```
final int SIZE = 10;
 for(int i = 0; i < SIZE; i++)
  myList[i] += 3;
```

- **Or using the `length` field, that contains the number of elements in array**

```
for(int i = 0; i < myList.length; i++)
  myList[i] += 3;
```

Declaring, creating, initializing Using the Shorthand Notation

- **Declare and create the array**

```
datatype[] arrayRefVar =
        {arrylist1, arrylist2,…};
```

- **Example**

```
double[] myList = {5.6, 4.5, 3.3,
13.2, 4, 34.33, 34, 45.45, 99.993,
11.3};
```

Declaring, creating, initializing Using the Shorthand Notation

```
double[] myList = {5.6, 4.5, 3.3, 13.2, 4, 34.33,
      34, 45.45, 99.993, 11.3};
```

This shorthand notation is equivalent to the following statements:

```
double[] myList = new double[10];
myList[0] = 5.6;  myList[1] = 4.5; myList[2] = 3.3;
myList[3] = 13.2; myList[4] = 4;   myList[5] = 34.33;
myList[6] = 34;   myList[7] = 45.45;
myList[8] = 99.993; myList[9] = 11.3;
```

CAUTION!!

Using the shorthand notation, you have to declare, create, and initialize the array all in one statement.

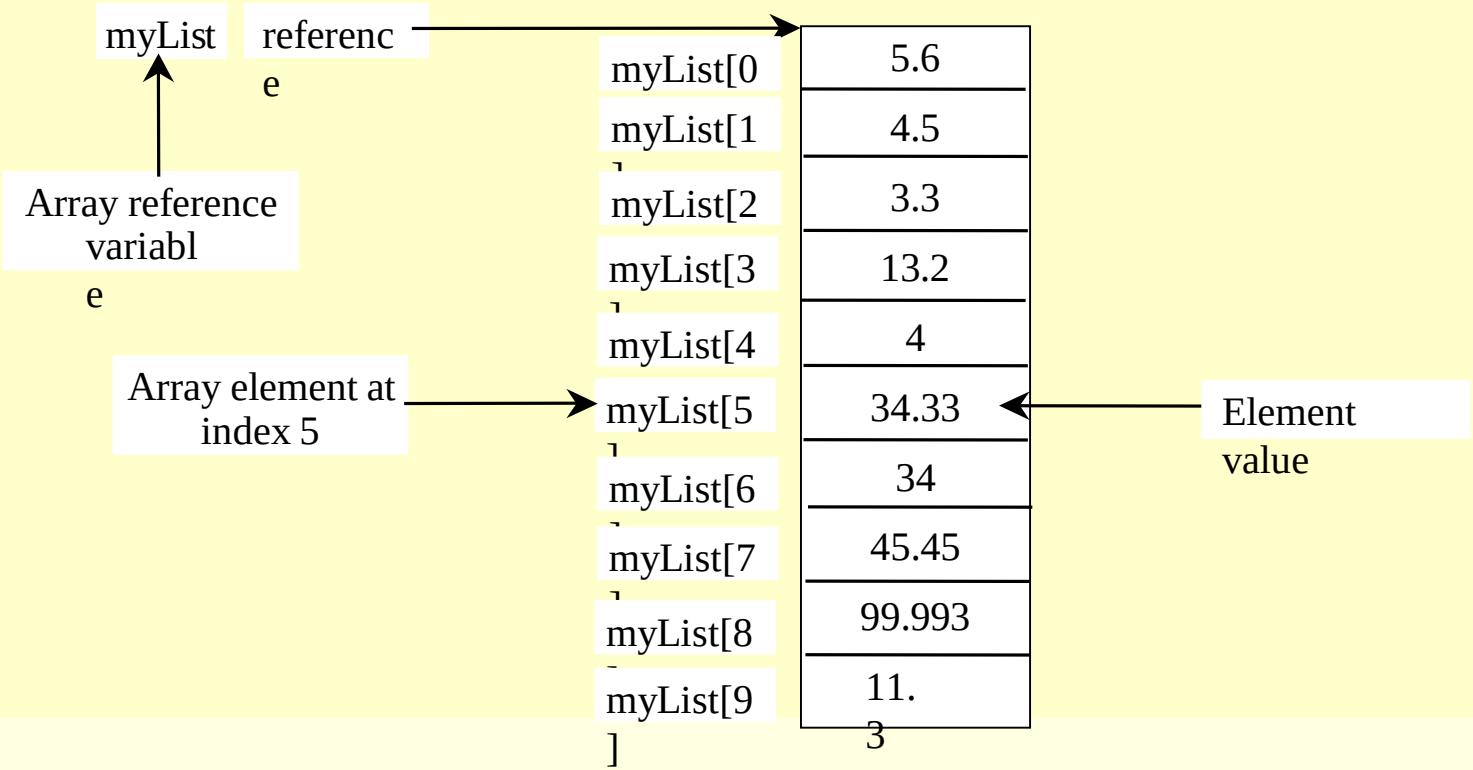
Splitting it would cause a syntax error. For example, the following is wrong:

```
double[] myList;
myList = {1.9, 2.9, 3.4, 3.5};
```

Arrays

- **Array's elements numbered beginning with zero**
  Can legally use any subscript from 0 through 9

```
double[] myList = new double[10];
```



Trace Program with Arrays

- Given the following program, what will be the output?

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

Trace Program with Arrays

Declare array variable values, create an array, and assign its reference to values

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = values[i] + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the array is created

0	0
1	0
2	0
3	0
4	0

Trace Program with Arrays

i becomes 1

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = values[i] + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the array is created

0	0
1	0
2	0
3	0
4	0

Trace Program with Arrays

i (=1) is less than 5

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = values[i] + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the array is created

0	0
1	0
2	0
3	0
4	0

Trace Program with Arrays

After this line is executed, value[1] is 1

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the first iteration

0	0
1	1
2	0
3	0
4	0

Trace Program with Arrays

After i++, i becomes 2

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = values[i] + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the first iteration

0	0
1	1
2	0
3	0
4	0

Trace Program with Arrays

i (= 2) is less than 5

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = values[i] + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the first iteration

0	0
1	1
2	0
3	0
4	0

Trace Program with Arrays

After this line is executed,
values[2] is 3 (2 + 1)

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the second iteration

0	0
1	1
2	3
3	0
4	0

Trace Program with Arrays

After this, i becomes 3.

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the second iteration

0	0
1	1
2	3
3	0
4	0

Trace Program with Arrays

i (=3) is still less than 5.

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the second iteration

0	0
1	1
2	3
3	0
4	0

Trace Program with Arrays

After this line, values[3] becomes 6 (3 + 3)

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the third iteration

0	0
1	1
2	3
3	6
4	0

Trace Program with Arrays

After this, i becomes 4

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the third iteration

0	0
1	1
2	3
3	6
4	0

Trace Program with Arrays

i (=4) is still less than 5

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the third iteration

0	0
1	1
2	3
3	6
4	0

Trace Program with Arrays

After this, values[4] becomes 10 (4 + 6)

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the fourth iteration

0	0
1	1
2	3
3	6
4	10

Trace Program with Arrays

After i++, i becomes 5

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the fourth iteration

0	0
1	1
2	3
3	6
4	10

Trace Program with Arrays

i (=5) < 5 is false. Exit the loop

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the fourth iteration

0	0
1	1
2	3
3	6
4	10

Trace Program with Arrays

After this line, values[0] is 11 (1 + 10)

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

0	11
1	1
2	3
3	6
4	10

Enhanced for Loop

JDK 1.5 and above introduced a new for loop that enables you to traverse the complete array sequentially without using an index variable. For example, the following code displays all elements in the array `scoreArray`:

```
for(int val : scoreArray)
      System.out.println(val);
```

In general, the syntax is

```
for (elementType value: arrayRefVar) {
    // Process the value
  }
```

You still have to use an index variable if you wish to traverse the array in a different order or change the elements in the array.

Array Processing

1. (Printing arrays)
2. (Summing all elements)
3. (Finding the largest element)
4. (Finding the smallest index of the largest element)

Example: Print an Array

- **Print an array of int and an array of double:**

```
public class PrintArrays  {
   static final int ARRSIZE = 10;        // The array's size

   static int intArr[] = new int[ARRSIZE];     // Create the int a
   static double realArr[] = { 1.1, 2.2, 3.3, 4.4,
         5.5, 6.6, 7.7, 8.8, 9.9, 10.10 };     // And a double array

   public static void main(String args[]) {
      System.out.println("Ints \t Reals");
      for (int k = 0; k < intArr.length; k++)
          System.out.println( intArr[k] + " \t " + realArr[k]);
    } // main()
} // PrintArrays
```

These must be static ...

... in order to refer to them in static main()

Uninitialized int array has default values of 0.

Program Output

Ints	Reals
0	1.1
0	2.2
0	3.3
0	4.4
0	5.5
0	6.6
0	7.7
0	8.8
0	9.9
0	10.1

Example: Store the First 100 Squares

```
public class Squares {
    static final int ARRSIZE = 100;            // The array's size
    static int intArr[] = new int[ARRSIZE];    // Create an int array

    public static void main(String args[]) {
        for (int k = 0; k < intArr.length; k++)  // Initialize the
array
            intArr[k] = (k+1) * (k+1);

        System.out.print("The first 100 squares are"); // Print a
heading
        for (int k = 0; k < intArr.length; k++)
        {       // Print the array
            if (k % 10 == 0)
                System.out.println(" ");      // 10 elements per row
            System.out.print( intArr[k] + " ");
        } // for
    } // main()
} // Squares
```

Program Output

```
The first 100 squares are
1 4 9 16 25 36 49 64 81 100
121 144 169 196 225 256 289 324 361 400
441 484 529 576 625 676 729 784 841 900
961 1024 1089 1156 1225 1296 1369 1444 1521 1600
1681 1764 1849 1936 2025 2116 2209 2304 2401 2500
2601 2704 2809 2916 3025 3136 3249 3364 3481 3600
3721 3844 3969 4096 4225 4356 4489 4624 4761 4900
5041 5184 5329 5476 5625 5776 5929 6084 6241 6400
6561 6724 6889 7056 7225 7396 7569 7744 7921 8100
8281 8464 8649 8836 9025 9216 9409 9604 9801
```

Exercise 1 (Question 1)

Modify program ArrayDemo and save as ArrayDemo1

- **Initialize the elements in one initialization statement**

```
class ArrayDemo {
    public static void main(String[] args) {
        int[] anArray; // declares an array of integers
        anArray = new int[10]; // allocates memory for 10 integers
        anArray[0] = 100; // initialize first element
        anArray[1] = 200; // initialize second element
        anArray[2] = 300; // etc.
        anArray[3] = 400;
        anArray[4] = 500;
        anArray[5] = 600;
        anArray[6] = 700;
        anArray[7] = 800;
        anArray[8] = 900;
        anArray[9] = 1000;
```

```
double[] myList = {5.6, 4.5, 3.3, 13.2, 4, 34.33,
        34, 45.45, 99.993, 11.3};
```

Exercise 1 (Question 1) cont.

1. **Use traditional for loop to print all the values**

```
System.out.println("Element at index 0: " + anArray[0]);
System.out.println("Element at index 1: " + anArray[1]);
System.out.println("Element at index 2: " + anArray[2]);
System.out.println("Element at index 3: " + anArray[3]);
System.out.println("Element at index 4: " + anArray[4]);
System.out.println("Element at index 5: " + anArray[5]);
System.out.println("Element at index 6: " + anArray[6]);
System.out.println("Element at index 7: " + anArray[7]);
System.out.println("Element at index 8: " + anArray[8]);
System.out.println("Element at index 9: " + anArray[9]);
}
}
```

```
class ArrayDemo1 {
    public static void main(String[] args) {
        int anArray[] = {100, 200, 300, 400,
          500, 600, 700, 800, 900, 1000};

        for (int i = 0; i < anArray.length; i++) {
            System.out.println("Element at
          index" + i + ": " + anArray[i]);
        }
    }
}
```

Exercise 1 (Question 2) cont.

Modify program ArrayDemo1 and save as ArrayDemo2

1. **Use enhanced for loop / for-each loop to print all the values**

```
class ArrayDemo2 {
    public static void main(String[] args) {
        int anArray[] = {100, 200, 300, 400,
            500, 600, 700, 800, 900, 1000};
        for (int i: anArray) {
            System.out.println(i);
        }
    }
}
```

- **The output from this program is:**

```
Element at index 0: 100
Element at index 1: 200
Element at index 2: 300
Element at index 3: 400
Element at index 4: 500
Element at index 5: 600
Element at index 6: 700
Element at index 7: 800
Element at index 8: 900
Element at index 9: 1000
```